ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Fintech Expense & Payment Platform

(исправленная и дополненная версия)

Стек: Java 17+ / Spring Boot / PostgreSQL / Redis / Kafka

Уровень: Middle+ / Senior Backend Engineer
Цель: подготовка к интервью в bigtech (Тинькофф, Яндекс, Revolut, Wise)

Об этом документе

Это переработанная версия оригинального ТЗ. Все исправления, дополнения и архитектурные замечания интегрированы в текст. Блоки с зелёной рамкой — добавления. Блоки с оранжевой рамкой — предупреждения. Блоки с красной рамкой — критические исправления ошибок оригинала.

1. Обзор проекта

1.1 Бизнес-контекст

Backend для финтех-платформы управления личными финансами. Система позволяет пользователям открывать счета, совершать переводы, отслеживать расходы и получать аналитику по тратам. Это не учебный CRUD — это продукт, за который платят деньги и где ошибка в коде = потеря реальных средств.

1.2 Что ты получишь

- Глубокое понимание ACID-транзакций и двойной записи (ledger) — ядро любого финтех-бэкенда
- Практику конкурентного программирования: pessimistic/optimistic locking, race conditions
- Опыт проектирования event-driven архитектуры с Kafka и outbox pattern
- Навыки оптимизации PostgreSQL: индексы, partitioning, EXPLAIN ANALYZE
- Продакшн-подход к REST API design, идемпотентности, пагинации
- Понимание распределённых систем: eventual consistency, retry, saga, circuit breaker
- Практику проектирования отказоустойчивых систем: fallback, graceful degradation
- Готовые ответы на вопросы собеседований уровня middle+/senior

1.3 Технологический стек

Компонент	Технология	Зачем
Язык	Java 17+	Records, sealed classes, pattern matching
Фреймворк	Spring Boot 3.x	DI, Web, Data, Security
БД (основная)	PostgreSQL 15+	ACID, partitioning, JSON, LISTEN/NOTIFY
Кеш / блокировки	Redis 7+	Distributed locks, кеш балансов, rate limiting, счётчики лимитов
Очереди событий	Apache Kafka	Event-driven, outbox pattern, ordering guarantees
Миграции	Flyway	Версионирование схемы БД, разделение по модулям
Отказоустойчивость	Resilience4j	Circuit breaker, retry, rate limiter, bulkhead
Тесты	JUnit 5 + Testcontainers	Интеграционные тесты с реальными БД
Мониторинг	Micrometer + Prometheus + Grafana	Метрики, JVM profiling, бизнес-дашборды
Трейсинг	Micrometer Tracing + Jaeger	Distributed tracing через HTTP и Kafka
Контейнеризация	Docker + docker-compose	Локальное окружение
Документация	SpringDoc / OpenAPI 3	Swagger UI

2. Архитектура системы

2.1 Модульный монолит (стартовая точка)

Начни с модульного монолита. Не бросайся сразу в микросервисы — на собесе тебя спросят "почему микросервисы?", и ответ "потому что модно" — это минус. Правильный ответ: "я начал с модульного монолита, выделил чёткие границы модулей, и готов к декомпозиции когда это оправдано нагрузкой".

Стратегия декомпозиции: когда выделять микросервисы

Оставить в монолите: auth, account, transaction, limits — тесно связаны, shared DB transaction критична для ACID. Разделение потребует distributed transaction (saga). Первый кандидат на выделение: analytics-service (тяжёлые read-запросы не должны конкурировать с write-нагрузкой). Второй: notification-service (внешние провайдеры, собственные retry). Третий: fraud-service (может потребовать ML, отдельный lifecycle). Триггер выделения: когда модуль потребляет >30% CPU основного инстанса.

Принципы модульного монолита для будущей декомпозиции

1. Модули общаются только через interfaces (Spring beans), никогда напрямую к репозиториям другого модуля.
1. Каждый модуль имеет свой @Configuration и свой набор таблиц. Нет cross-module JOINs.
2. Асинхронная коммуникация через ApplicationEventPublisher — при переходе к микросервисам заменяется на Kafka без изменения бизнес-логики.
3. Flyway migrations разделены по модулям: V1.001__auth_create_users.sql, V2.001__account_create_accounts.sql.

2.2 Модули системы

Изменение vs оригинал

Выделен отдельный limits-module (из account-module) — проверка лимитов это cross-cutting concern. Категории (category) понижены до sub-package внутри transaction. Добавлен infrastructure-слой для idempotency, outbox, rate limiting.

Модуль	Ответственность	Ключевые сущности
auth-module	Регистрация, логин, JWT (RS256) + refresh tokens, token family rotation	User, RefreshToken, TokenFamily
account-module	Счета, балансы, холды (отдельная таблица holds)	Account, AccountBalance, Hold
limits-module	Дневные/месячные лимиты на расходы, Redis-счётчики	SpendingLimit, DailyCounter
transaction-module	Переводы, ledger, категории расходов, автокатегоризация	Transaction, LedgerEntry, Category, CategoryRule
analytics-module	Отчёты, агрегации, витрины данных (Kafka consumer)	DailyExpenseSummary, MonthlyReport
notification-module	Уведомления: push, email, in-app (роутинг по каналам)	Notification, NotificationPreference, Channel
fraud-module	Правила обнаружения подозрительных операций (асинхронно)	FraudRule, FraudAlert, SeverityLevel
audit-module	Неизменяемый лог всех действий (не только денежных)	AuditEntry
infrastructure	Idempotency filter, outbox poller, rate limiter, reconciliation	IdempotencyKey, OutboxEvent

2.3 Коммуникация между модулями

Внутри монолита модули общаются через Spring Application Events (синхронно для критичных операций, асинхронно для аналитики). При переходе к микросервисам эти события заменяются на Kafka-топики без изменения бизнес-логики.

Ключевые доменные события:

- **TransactionCreatedEvent** — триггерит обновление аналитики, проверку фрода, уведомление, автокатегоризацию
- **AccountBalanceChangedEvent** — триггерит проверку лимитов, обновление Redis-кеша баланса
- **FraudAlertCreatedEvent** — триггерит уведомление. Блокировка счёта ТОЛЬКО при severity=HIGH (не автоматически!)
- **AuditEvent** — записывает любое значимое действие в audit log

Критическое исправление: FraudAlert не должен автоматически блокировать счёт

В оригинальном T3 FraudAlertCreatedEvent напрямую триггерит блокировку. Это опасно: ложное срабатывание заблокирует пользователя. Реализуй severity-based подход: HIGH — блокировка + уведомление, MEDIUM — только уведомление + лог, LOW — только лог. Блокировка по HIGH должна требовать confirmation от fraud-review процесса.

3. Схема базы данных (PostgreSQL)

Общее замечание по UUID

Используй UUID v7 (timestamp-sortable) вместо gen_random_uuid() (v4). UUID v4 создаёт случайные значения, что приводит к фрагментации B-tree индексов и увеличению WAL traffic. UUID v7 монотонно возрастает, что оптимально для индексов. В PostgreSQL 17+ есть встроенная функция uuidv7(). Для более ранних версий — pg_uuidv7 extension или генерация на стороне Java (uuid-creator library).

3.1 Таблица: users

```
CREATE TABLE users (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    email           VARCHAR(255) NOT NULL UNIQUE,
    password_hash   VARCHAR(255) NOT NULL,
    full_name       VARCHAR(255) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    -- GDPR: при запросе на удаление обнуляем PII, ставим ANONYMIZED
    anonymized_at   TIMESTAMP WITH TIME ZONE,
    created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now()
);
```

Добавлено: поддержка GDPR

Поле anonymized_at + статус ANONYMIZED. При GDPR-запросе на удаление: обнулить email (заменить на hash), full_name, password_hash. Не удалять строку — это нарушит целостность ledger. Status = 'ANONYMIZED' блокирует логин и все операции.

3.2 Таблица: accounts

```
CREATE TABLE accounts (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    user_id         UUID NOT NULL REFERENCES users(id),
    name            VARCHAR(100) NOT NULL,
    currency        VARCHAR(3) NOT NULL DEFAULT 'USD',
    balance         DECIMAL(19,4) NOT NULL DEFAULT 0,
    hold_amount     DECIMAL(19,4) NOT NULL DEFAULT 0, -- денормализованная сумма из
holds
    status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',
    version         BIGINT NOT NULL DEFAULT 0,  -- optimistic locking
    blocked_at      TIMESTAMP WITH TIME ZONE,
    block_reason    VARCHAR(500),
    created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    updated_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    CONSTRAINT positive_balance CHECK (balance >= 0),
    CONSTRAINT positive_hold CHECK (hold_amount >= 0),
    CONSTRAINT available_balance CHECK (balance >= hold_amount)
);


CREATE INDEX idx_accounts_user_id ON accounts(user_id);
```

3.3 Таблица: holds

Новая таблица (отсутствовала в оригинале)

Оригинал хранил hold_amount как единственное поле. В реальности холдов может быть несколько (pending операции, авторизации). Отдельная таблица holds с accounts.hold_amount как денормализованная сумма.

```
CREATE TABLE holds (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    account_id      UUID NOT NULL REFERENCES accounts(id),
    transaction_id  UUID, -- связь с транзакцией, если hold из-за перевода
    amount          DECIMAL(19,4) NOT NULL,
    reason          VARCHAR(100) NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'ACTIVE', -- ACTIVE, RELEASED,
CAPTURED
    expires_at      TIMESTAMP WITH TIME ZONE NOT NULL,
    created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now()
);

CREATE INDEX idx_holds_active ON holds(account_id, expires_at) WHERE status = 'ACTIVE';
```

3.4 Таблица: transactions (партиционированная)

```
CREATE TABLE transactions (
    id                UUID NOT NULL DEFAULT uuidv7(),
    idempotency_key   VARCHAR(255) NOT NULL,
    source_account_id UUID,
    target_account_id UUID,
    amount            DECIMAL(19,4) NOT NULL,
    currency          VARCHAR(3) NOT NULL,
    type              VARCHAR(30) NOT NULL, -- TRANSFER, DEPOSIT, WITHDRAWAL
    status            VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    description       VARCHAR(500),
    category_id       UUID,
    created_at        TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    completed_at      TIMESTAMP WITH TIME ZONE,
    PRIMARY KEY (id, created_at) -- составной PK для partitioning
) PARTITION BY RANGE (created_at);

-- Уникальность idempotency_key глобально
CREATE UNIQUE INDEX idx_transactions_idempotency ON transactions(idempotency_key);
CREATE INDEX idx_transactions_source ON transactions(source_account_id, created_at
DESC);
CREATE INDEX idx_transactions_target ON transactions(target_account_id, created_at
DESC);
CREATE INDEX idx_transactions_pending ON transactions(created_at) WHERE status =
'PENDING';
CREATE INDEX idx_transactions_analytics
    ON transactions(source_account_id, category_id, created_at DESC)
    WHERE type IN ('TRANSFER', 'WITHDRAWAL');
```

Критическое изменение: FK убраны из partitioned table

PostgreSQL не поддерживает FK из partitioned таблицы на обычные таблицы. Попытка создать REFERENCES accounts(id) в partitioned transactions приведёт к ошибке. Целостность

обеспечивается на уровне приложения. Для финтеха это стандартная практика — ledger-based accounting не зависит от FK.

Автоматическое создание партиций: использовать pg_partman или cron job. Без автоматизации система сломается 1-го числа следующего месяца при INSERT в несуществующую партицию.

```
-- Пример создания партиции
CREATE TABLE transactions_2026_04
    PARTITION OF transactions
    FOR VALUES FROM ('2026-04-01') TO ('2026-05-01');

-- Архивация: партиции старше 1 года отсоединить и перенести в cold storage
ALTER TABLE transactions DETACH PARTITION transactions_2025_01;
```

3.5 Таблица: ledger_entries (двойная запись)

Каждая транзакция порождает минимум 2 записи: дебет и кредит. Сумма всех записей по счёту = актуальный баланс. Это стандарт финтеха.

```
CREATE TABLE ledger_entries (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    transaction_id  UUID NOT NULL,
    account_id      UUID NOT NULL,
    entry_type      VARCHAR(10) NOT NULL, -- DEBIT, CREDIT
    amount          DECIMAL(19,4) NOT NULL,
    balance_after   DECIMAL(19,4) NOT NULL,
    created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now()
) PARTITION BY RANGE (created_at); -- тоже партиционируем, растёт 2x быстрее

CREATE INDEX idx_ledger_account ON ledger_entries(account_id, created_at DESC);
CREATE INDEX idx_ledger_transaction ON ledger_entries(transaction_id);
CREATE INDEX idx_ledger_reconciliation ON ledger_entries(account_id, entry_type);
```

Добавлено: партиционирование ledger_entries

В оригинале ledger_entries не партиционирована, хотя растёт вдвое быстрее transactions (2 записи на транзакцию). Без partitioning индексы станут неуправляемыми при >100M строк.

3.6 Таблица: idempotency_keys

Новая таблица (критическое исправление)

В оригинале idempotency_key хранится в transactions. Проблема: если транзакция упала до COMMIT — ключ не сохранён, retry создаст дубликат. Отдельная таблица решает race condition через INSERT ON CONFLICT.

```
CREATE TABLE idempotency_keys (
    key         VARCHAR(255) PRIMARY KEY,
    response    JSONB,                          -- результат первого выполнения
    status      VARCHAR(20) NOT NULL DEFAULT 'PROCESSING', -- PROCESSING, COMPLETED,
FAILED
    created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    expires_at  TIMESTAMP WITH TIME ZONE NOT NULL -- чистить ключи старше 24-48 часов
);

CREATE INDEX idx_idempotency_expires ON idempotency_keys(expires_at);
```

3.7 Таблица: outbox_events

Outbox pattern — гарантия доставки событий. Событие записывается в ту же транзакцию, что и бизнес-данные. Отдельный процесс читает outbox и публикует в Kafka.

```
CREATE TABLE outbox_events (
    id              UUID PRIMARY KEY DEFAULT uuidv7(),
    aggregate_type  VARCHAR(100) NOT NULL,
    aggregate_id    VARCHAR(255) NOT NULL,
    event_type      VARCHAR(100) NOT NULL,
    payload         JSONB NOT NULL,
    status          VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at      TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now(),
    published_at    TIMESTAMP WITH TIME ZONE
);

-- Partial index: только PENDING события для polling
CREATE INDEX idx_outbox_pending ON outbox_events(status, created_at) WHERE status =
'PENDING';
-- Мониторинг лага
CREATE INDEX idx_outbox_lag ON outbox_events(created_at) WHERE status = 'PENDING';
```

Очистка: НЕ партиционировать (данные короткоживущие). Batch DELETE обработанных событий старше 7 дней по cron.

3.8 Таблица: audit_log

> **Новая таблица (отсутствовала в оригинале)**
> Для финтеха обязателен неизменяемый audit trail ВСЕХ действий, не только денежных. Ledger — audit для денег, но нет записей о: смене пароля, изменении лимитов, блокировке/разблокировке, dismiss fraud alert.

```
CREATE TABLE audit_log (
    id          UUID PRIMARY KEY DEFAULT uuidv7(),
    user_id     UUID NOT NULL,
    action      VARCHAR(100) NOT NULL, -- LOGIN, PASSWORD_CHANGE, LIMIT_UPDATE,
ACCOUNT_BLOCK, ...
    entity_type VARCHAR(50) NOT NULL,  -- USER, ACCOUNT, TRANSACTION, FRAUD_ALERT
    entity_id   UUID,
    details     JSONB,                 -- old/new values, IP, User-Agent
    ip_address  INET,
    created_at  TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT now()
) PARTITION BY RANGE (created_at);

CREATE INDEX idx_audit_user ON audit_log(user_id, created_at DESC);
CREATE INDEX idx_audit_entity ON audit_log(entity_type, entity_id, created_at DESC);
```

3.9 Дополнительные таблицы

Реализуй самостоятельно по аналогии: categories, category_rules, daily_expense_summary (партиционированная по месяцам), fraud_rules, fraud_alerts (с полем severity: HIGH/MEDIUM/LOW), notifications, notification_preferences, refresh_tokens (с token_family_id), spending_limits.

3.10 Требования к БД (это спрашивают на собесах)

- **Partitioning:** transactions, ledger_entries, audit_log, daily_expense_summary — по created_at (RANGE по месяцам). Обязательно автоматизировать создание партиций через pg_partman.
- **EXPLAIN ANALYZE:** для каждого критичного запроса. Ты должен уметь читать план выполнения и объяснять index scan vs seq scan, partition pruning.
- **Денормализация:** daily_expense_summary — денормализация для аналитики. accounts.hold_amount — денормализация суммы из holds. Объясни trade-off.
- **Составные индексы:** для частых запросов. Partial indexes (WHERE status = 'PENDING') для outbox и transactions.
- **VACUUM tuning:** для горячих таблиц (transactions, ledger_entries) настрой autovacuum: autovacuum_vacuum_cost_delay = 2ms, autovacuum_vacuum_scale_factor = 0.01.
- **UUID v7:** вместо v4 для всех PK. Монотонный рост оптимизирует B-tree индексы.

4. REST API (контракт)

4.1 Auth API

POST /api/v1/auth/register

Регистрация нового пользователя.

Request Body:

```
{ "email": "user@example.com",
  "password": "SecureP@ss123",
  "fullName": "Иван Петров" }
```

Response 201 Created:

```
{ "data": { "id": "550e8400-...", "email": "user@example.com",
    "fullName": "Иван Петров", "status": "ACTIVE",
    "createdAt": "2026-04-15T10:30:00Z" } }
```

Response 409 Conflict (email уже существует):

```
{ "error": { "code": "EMAIL_ALREADY_EXISTS",
    "message": "User with this email already exists" } }
```

Валидация: email — формат RFC 5322; password — min 8 символов, 1 заглавная, 1 цифра, 1 спецсимвол; fullName — 2-255 символов.

POST /api/v1/auth/login

Аутентификация, возвращает пару токенов.

Response 200 OK:

```
{ "data": { "accessToken": "eyJhbGciOiJSUzI1NiIs...",
    "refreshToken": "dGhpcyBpcyBh...",
    "expiresIn": 900, "tokenType": "Bearer" } }
```

> **Добавлено: RS256 вместо HS256**
> Алгоритм подписи JWT — RS256 (асимметричный). Позволяет верифицировать токен без секрета (нужен только public key). При переходе к микросервисам каждый сервис верифицирует токен самостоятельно без обращения к auth-сервису.

Заголовки: accessToken живёт 15 мин, refreshToken — 30 дней. После 5 неудачных попыток — блокировка на 15 мин (rate limiting через общий middleware, не хардкод в auth).

POST /api/v1/auth/refresh

Обновление access token по refresh token. Старый refresh token инвалидируется (rotation).

> **Добавлено: Token Family**
> Реализуй token family: каждая цепочка refresh tokens имеет family_id. Если refresh token использован дважды (оригинальным владельцем и злоумышленником), инвалидируй ВСЮ цепочку (все токены с этим family_id). Храни family_id в таблице refresh_tokens.

POST /api/v1/auth/logout

Инвалидация refresh token + добавление access token JTI в Redis blacklist.

> **Добавлено: Redis blacklist для access tokens**
> Без blacklist после logout access token живёт ещё до 15 минут. Для финтеха это неприемлемо. При logout: добавить JTI (JWT ID) токена в Redis SET с TTL = оставшееся время жизни. При каждом запросе проверять JTI в blacklist.

4.2 Accounts API

Все эндпоинты требуют заголовок Authorization: Bearer {accessToken}. Пользователь видит только свои счета.

Эндпоинты: POST /api/v1/accounts (создание, max 10 на пользователя), GET /api/v1/accounts (список), GET /api/v1/accounts/{id} (детали + лимиты), PATCH /api/v1/accounts/{id}/block (блокировка), PUT /api/v1/accounts/{id}/limits (установка лимитов).

Важно: GET /accounts/{id} для несуществующего или чужого счёта — всегда 404 (не раскрываем факт существования чужого счёта). availableBalance = balance - holdAmount.

4.3 Transactions API

POST /api/v1/transactions

Создание перевода между счетами. Главный эндпоинт системы.

Request Headers:

```
Authorization: Bearer {accessToken}
X-Idempotency-Key: 7c9e6679-7425-40de-944b-e07fc1f90ae7
```

Request Body:

```
{ "sourceAccountId": "a1b2c3d4-...",
  "targetAccountId": "b2c3d4e5-...",
  "amount": "250.00", "currency": "USD",
  "description": "Оплата за обед",
  "categoryId": "c3d4e5f6-..." }
```

Response 201 Created: возвращает транзакцию с ledgerEntries (DEBIT + CREDIT).

Response 409 Conflict: дубликат idempotency key — возвращает результат первого выполнения.

Коды ошибок: INSUFFICIENT_FUNDS (422), ACCOUNT_BLOCKED (403), DAILY_LIMIT_EXCEEDED (422), SAME_ACCOUNT (422), CURRENCY_MISMATCH (422), ACCOUNT_NOT_FOUND (404).

> **Добавлено: Rate limiting на transaction endpoint**
> В оригинале rate limiting указан только для auth. Для POST /transactions обязателен rate limit: 100 req/min per user. Без него один пользователь может DDoS-ить систему. Реализация: Bucket4j + Redis для distributed rate limiting.

GET /api/v1/transactions

История транзакций с cursor-based пагинацией и фильтрами.

Query Params: accountId (required), cursor (optional), limit (default 20, max 100), type, dateFrom, dateTo, categoryId.

Cursor-based pagination: cursor кодирует (createdAt, id) последней записи. Стабильная пагинация даже при вставке новых данных. Offset плохо масштабируется (skip N строк = seq scan).

4.4 Categories API

POST /api/v1/categories — создание категории. GET /api/v1/categories — список с суммой трат за месяц (query param month=YYYY-MM).

4.5 Analytics API

GET /api/v1/analytics/expenses — расходы по категориям за период. GET /api/v1/analytics/daily — дневная разбивка (для графиков). GET /api/v1/analytics/monthly-report — полный месячный отчёт: расходы, доходы, баланс, сравнение с прошлым месяцем.

Предупреждение: eventual consistency

Аналитика обновляется асинхронно. Когда пользователь делает перевод и сразу смотрит отчёт — он может не увидеть последнюю операцию. Это нормально и это eventual consistency. Допустимо для аналитики, недопустимо для баланса. На собесе расскажи trade-off.

4.6 Fraud Alerts API

GET /api/v1/fraud-alerts — список подозрительных операций (фильтр по status: OPEN/RESOLVED/DISMISSED). PATCH /api/v1/fraud-alerts/{id}/dismiss — пометить как ложное срабатывание.

4.7 Общие стандарты API

- **Единый формат ошибок:** { "error": { "code": "...", "message": "...", "details": {...} } }
- **Существительные для ресурсов:** /transactions, /accounts — не /makeTransfer, /getBalance
- **Версионирование:** /api/v1/ — расскажи зачем и какие стратегии бывают (URL, header, media type)
- **Суммы как строки:** "250.0000" — не как float (потеря точности при JSON-сериализации)
- **Даты в ISO 8601 UTC:** "2026-04-15T11:15:00Z"
- **Обёртка ответа:** { data: ..., meta?: ... } для консистентности

Код	Когда	Пример
200	Успешное чтение или обновление	GET /accounts, PATCH /block
201	Ресурс создан	POST /transactions
204	Успех без тела ответа	POST /auth/logout
400	Невалидный JSON / синтаксис	Битый JSON в body
401	Не аутентифицирован	Нет токена или невалидный
403	Нет прав	Перевод с заблокированного счёта
404	Ресурс не найден	Несуществующий ID
409	Конфликт	Дубликат idempotency key
422	Бизнес-ошибка	Недостаточно средств, лимит
429	Rate limit	Слишком много запросов
500	Внутренняя ошибка	Непредвиденный сбой

5. Бизнес-логика (критические кейсы)

5.1 Перевод между счетами (главная фича)

Полный алгоритм перевода (исправленный):

2. Валидация входных данных (сумма > 0, счета существуют, разные счета, валюта совпадает)
4. Проверка idempotency: INSERT INTO idempotency_keys ON CONFLICT DO NOTHING. Если affected rows = 0 — вернуть прошлый результат
5. Проверка rate limit (Redis: 100 req/min per user)
6. Проверка лимитов (Redis счётчики daily_spent / monthly_spent)
7. Начало БД-транзакции
8. SELECT ... FOR UPDATE — блокировка ОБА счёта В ПОРЯДКЕ ВОЗРАСТАНИЯ UUID (предотвращение deadlock!)
9. Проверка баланса: balance - hold_amount >= amount
10. Списание со счёта-отправителя (UPDATE accounts SET balance = balance - amount)
11. Зачисление на счёт-получателя
12. Создание записи в transactions (статус COMPLETED)
13. Создание 2 записей в ledger_entries (DEBIT + CREDIT)
14. Сохранение события в outbox_events
15. COMMIT транзакции
16. Обновление Redis: баланс, счётчики лимитов (за пределами DB tx — если Redis упал, это не откатывает перевод)

Критическое исправление: порядок блокировки счетов

В оригинале не указан порядок блокировки. При переводе A->B и одновременном B->A без фиксированного порядка — deadlock. Решение: ВСЕГДА блокировать счета в порядке возрастания UUID. Код: List<UUID> ordered = Stream.of(sourceId, targetId).sorted().toList(); затем SELECT ... FOR UPDATE WHERE id IN (:ordered) ORDER BY id.

Критическое исправление: scope DB-транзакции

DB-транзакция должна быть МИНИМАЛЬНОЙ: заблокировать -> списать/зачислить -> ledger -> outbox -> COMMIT. Всё остальное (проверка лимитов, уведомления, fraud, аналитика) — за пределами DB-транзакции, event-driven. Иначе блокировки строк account живут слишком долго при нагрузке.

5.2 Race condition — реализуй оба варианта

Pessimistic locking

```
SELECT * FROM accounts WHERE id = :id FOR UPDATE;
```

Плюсы: простота, предсказуемость. **Минусы:** блокирует строку, плохо масштабируется для горячих аккаунтов.

Optimistic locking

```
UPDATE accounts SET balance = balance - :amount, version = version + 1
WHERE id = :id AND version = :expectedVersion;
-- Если affected rows = 0 -> retry
```

Когда что использовать

Pessimistic — для transfer endpoint (высококонкурентный, критичный). Optimistic — когда конфликты редки (обновление лимитов, настроек). Для горячих аккаунтов (мерчант, >10 TPS) — только pessimistic. При optimistic locking с горячим аккаунтом — 99% retry, starvation.

5.3 Идемпотентность (исправленная)

Клиент генерирует UUID и передаёт в заголовке X-Idempotency-Key. Сервер проверяет через отдельную таблицу idempotency_keys (не через transactions — см. секцию 3.6).

Алгоритм проверки:

```
INSERT INTO idempotency_keys (key, status, expires_at)
VALUES (:key, 'PROCESSING', now() + interval '24 hours')
ON CONFLICT (key) DO NOTHING;

-- affected rows = 0 -> ключ уже существует
-- -> SELECT response FROM idempotency_keys WHERE key = :key
-- -> если status = 'PROCESSING' -> конкурентный запрос, подождать/retry
-- -> если status = 'COMPLETED' -> вернуть сохранённый response

-- affected rows = 1 -> мы первые, выполнить операцию
-- после успеха: UPDATE idempotency_keys SET status='COMPLETED', response=:json
```

5.4 Лимиты на расходы

- Пользователь устанавливает дневной/месячный лимит на счёт
- Перед каждым переводом проверяй: сумма переводов за период + текущий перевод <= лимит
- Оптимизация: Redis counter с TTL (INCRBY при каждой транзакции)
- Fallback: если Redis недоступен — SQL SUM() (медленнее, но работает)
- Ночной batch-job пересчитывает все счётчики из БД (исправление drift)

5.5 Fraud detection (правила + severity)

Правило	Severity	Действие
Сумма > 10x средней за 30 дней	HIGH	Уведомление + review (НЕ автоблокировка)
> 10 переводов за 1 минуту	HIGH	Уведомление + временная блокировка отправки
Перевод на новый счёт > $5000	MEDIUM	Уведомление пользователю
Перевод в нерабочее время (2:00-5:00)	LOW	Лог для анализа

Оптимизация fraud: предвычисление средней

Правило '10x средней за 30 дней' требует вычисления на каждую транзакцию. При 1M пользователей это убьёт БД. Решение: предвычислять в Redis (ключ fraud:avg30d:{accountId}), обновлять скользящим средним, полный пересчёт batch-ом ночью. Правило '>10 за минуту': Redis sorted set (ZADD по timestamp, ZRANGEBYSCORE для подсчёта).

6. Event-Driven архитектура

6.1 Outbox Pattern

Почему нельзя просто отправить событие в Kafka после коммита? Потому что возможен сценарий: коммит прошёл, но отправка в Kafka упала — событие потеряно. Outbox решает это:

3. В той же БД-транзакции пишем событие в таблицу outbox_events
17. Отдельный поток (OutboxPoller) читает PENDING-события и публикует в Kafka
18. После успешной публикации помечаем событие как PUBLISHED
19. Consumers должны быть идемпотентными (одно событие может прийти дважды — at-least-once)

Outbox Polling: реализация

```
-- SKIP LOCKED: каждый инстанс берёт свой batch без ожидания
SELECT * FROM outbox_events
WHERE status = 'PENDING'
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Polling interval: 100ms. **Batch size:** 100 events.

> **Эволюция: polling -> CDC**
> Начать с polling (простота). Переходить на Debezium CDC когда outbox polling станет >=5% нагрузки на DB. CDC читает WAL, минимальная нагрузка на БД, latency <100ms. Промежуточный вариант: PostgreSQL LISTEN/NOTIFY для мгновенного wakeup poller-а + polling как fallback.

6.2 Kafka Topics

Топик	Producer	Consumers	Partition key
transaction.created	Transaction Module	Analytics, Fraud, Notifications, AutoCategory	source_account_id
account.balance.changed	Account Module	Limits, Analytics	account_id
fraud.alert.created	Fraud Module	Notifications (+ Account при HIGH)	account_id
notification.send	Notification Router	Email/Push/InApp workers	user_id
audit.event	All Modules	Audit Writer	user_id

> **Критично: партиционирование по account_id**
> Партиционировать по account_id, НЕ по transaction_id. Это гарантирует ordering всех событий одного счёта в одной партиции. Без этого consumer может обработать events out of order и получить некорректный баланс в витрине. Число партиций: 12 (кратно 3-4-6 инстансам). Увеличивать нельзя без repartitioning — закладывать с запасом.

6.3 Eventual Consistency

Аналитика обновляется асинхронно. Это допустимо для аналитики, но недопустимо для баланса. На собесе: объясни почему баланс — strong consistency (пользователь не может потратить больше, чем есть), а аналитика — eventual (задержка в отчёте не критична).

7. Кэширование (Redis)

Новый раздел (отсутствовал в оригинале)
Оригинальное T3 упоминает Redis для кеша и блокировок, но не детализирует стратегию кэширования. Это критический пробел для финтех-системы.

7.1 Что кэшировать

Данные	Ключ Redis	TTL	Стратегия
Баланс (для отображения)	balance:{accountId}	30 сек	Write-through при каждой транзакции
Дневной расход (лимиты)	daily_spent:{accountId}:{date}	До конца дня	INCRBY при транзакции, batch пересчёт ночью
Месячный расход	monthly_spent:{accountId}:{YYYY-MM}	До конца месяца	Аналогично
Средняя 30 дней (фрод)	fraud:avg30d:{accountId}	24h	Batch пересчёт + инкремент real-time
JWT blacklist	jwt:blacklist:{jti}	= TTL токена	SET при logout
Rate limit	ratelimit:{userId}:{endpoint}	1 мин	Sliding window (ZADD)
Категории пользователя	categories:{userId}	10 мин	Invalidate при CRUD
Idempotency (fast check)	idempotency:{key}	24h	SET при первом выполнении

7.2 Что НЕ кэшировать

- **Баланс для списания.** Кэшированный баланс — для отображения (GET /accounts). Для операции списания — всегда SELECT FROM DB с FOR UPDATE. Иначе race condition.
- **Ledger entries.** Immutable, запрашиваются редко, кэш не окупится.
- **Транзакции пользователя.** Cursor-based пагинация делает кэширование бессмысленным.

7.3 Отказоустойчивость Redis

Принцип: система ДОЛЖНА работать без Redis (деградация, не отказ). Redis — оптимизация, а не зависимость.

- Redis down — лимиты проверять через SQL (медленнее, но работает)
- Redis down — баланс показывать из DB
- Redis down — rate limiting пропускать (лучше пропустить лимит, чем отказать в обслуживании)
- Redis down — JWT blacklist: сократить TTL access token до 1 мин через application config
- Circuit breaker (Resilience4j): 5 failures / 10 sec — open circuit на 30 сек — DB fallback

8. Конкурентный доступ — проблемные места

Новый раздел (отсутствовал как отдельная тема в оригинале)
Оригинал описывает pessimistic/optimistic locking, но не перечисляет все race conditions в системе и их решения.

8.1 Двойное списание (primary risk)

Сценарий: два запроса на перевод с одного счёта одновременно. Оба проверяют баланс (100$), оба видят достаточно средств, оба списывают — баланс -100$. **Решение:** SELECT ... FOR UPDATE. Second line of defense: CHECK constraint balance >= 0.

8.2 Deadlock при встречных переводах

Сценарий: перевод A->B и одновременный B->A. Thread 1 блокирует A, Thread 2 блокирует B. Оба ждут друг друга. **Решение:** блокировать счета в порядке возрастания UUID.

8.3 Idempotency key race condition

Сценарий: два запроса с одним key одновременно (retry с клиента). Оба проверяют — ключа нет. Оба начинают выполнение. **Решение:** INSERT ON CONFLICT DO NOTHING. affected rows = 0 — ключ уже взят.

8.4 Outbox poller — duplicate processing

Сценарий: два инстанса приложения берут одно PENDING событие. **Решение:** SELECT ... FOR UPDATE SKIP LOCKED.

8.5 Redis counter drift

Сценарий: daily_spent в Redis рассинхронизирован (Redis упал, INCRBY не дошёл). **Решение:** ночной batch-job пересчёт из БД, при miss в Redis — fallback к SQL SUM(). Redis — soft limit, жёсткая проверка — в DB transaction.

8.6 Hot account (мерчант-получатель)

Сценарий: популярный мерчант получает тысячи платежей/сек. FOR UPDATE на его строке — contention. **Решения:** 1) Асинхронные зачисления с batch-обновлением баланса. 2) Шардинг горячего аккаунта на несколько sub-accounts. Это самое сложное узкое место системы.

9. Отказоустойчивость

Новый раздел (отсутствовал в оригинале)
Оригинал упоминает retry и saga в контексте 'что рассказать на собесе', но не описывает конкретные стратегии. Для финтех-системы это критический пробел.

9.1 Retry strategy

Операция	Max retries	Backoff	Условие retry
DB transaction (optimistic)	3	50ms, 100ms, 200ms	OptimisticLockException
Kafka publish (outbox)	Infinite (poller)	100ms interval	Любая ошибка
Redis read	1	0 (immediate)	TimeoutException → fallback к DB
Redis write	1	0	Failure → лог warning, не блокировать
Notification send	5	Exp: 1s, 5s, 25s, 2m, 10m	Transient error
Fraud check	0	—	Failure → лог, не блокировать перевод

9.2 Circuit Breaker (Resilience4j)

Зависимость	Failure threshold	Wait duration	Fallback
Redis	5 failures / 10 sec	30 sec	DB fallback для чтения, skip для записи
Kafka (N/A — outbox)	—	—	Outbox таблица — встроенный буфер
Notification provider	10 / 1 min	60 sec	Сохранить в DB, retry позже
PostgreSQL replica	3 / 10 sec	30 sec	Redirect на primary (с увеличенным timeout)

9.3 Fallback при отказе компонентов

- **Redis down:** баланс из DB, лимиты через SQL, rate limiting пропускать, JWT blacklist — сократить TTL.
- **Kafka down:** outbox pattern — события накапливаются в DB. При восстановлении Kafka — poller вычитает. Данные не теряются. Alert при outbox lag > 5 мин.
- **PostgreSQL replica down:** аналитика — 503 или переключение на primary.
- **PostgreSQL primary down:** всё встаёт. Automatic failover через Patroni/pg_auto_failover. RTO: <30 сек.

9.4 Graceful shutdown

Добавлено: предотвращение потери in-flight транзакций
При деплое (rolling update) необходим graceful shutdown: 1) Перестать принимать новые HTTP-запросы. 2) Дождаться завершения текущих DB-транзакций (timeout 30 сек). 3) Дождаться flush outbox poller. 4) Закрыть Kafka producer. 5) Закрыть DB connections. Spring Boot shutdown hooks + kubernetes preStop hook.

10. Мониторинг и логирование

Раздел расширен
Оригинал ограничивается упоминанием Micrometer + Prometheus. Добавлены конкретные метрики, правила логирования, distributed tracing.

10.1 Business metrics (Prometheus)

- **transactions_total** (counter, labels: type, status, currency) — количество транзакций
- **transactions_amount_total** (counter, labels: type, currency) — объём в деньгах
- **transaction_duration_seconds** (histogram, p50/p95/p99) — latency перевода
- **fraud_alerts_total** (counter, labels: severity, rule)
- **daily_active_users** (gauge)

10.2 Infrastructure metrics

- **outbox_lag_events** (gauge) — количество PENDING в outbox. Alert: > 100
- **outbox_lag_seconds** (gauge) — возраст старейшего PENDING. Alert: > 60s
- **db_connection_pool_active** (gauge) — HikariCP connections. Alert: > 80% max
- **redis_command_duration_seconds** (histogram)
- **kafka_consumer_lag** (gauge per partition). Alert: > 10000
- **jvm_gc_pause_seconds** (histogram). Alert: p99 > 200ms

10.3 Что обязательно логировать (structured JSON)

Каждый лог-entry содержит: traceId, userId, accountId, requestId.

INFO level

- Каждый перевод: сумма, source, target, статус, duration_ms
- Каждый логин: userId, IP, User-Agent, success/failure
- Каждая блокировка/разблокировка счёта: reason, who triggered
- Каждый fraud alert: rule, severity, transaction details
- Outbox events: published/failed, lag
- Rate limit triggered: userId, endpoint

WARN level

- Redis connection failure / fallback to DB
- Kafka publish failure (outbox fallback)
- Optimistic lock conflict / retry
- Slow query > 100ms
- Balance reconciliation mismatch

ERROR level

- DB connection failure
- Uncaught exceptions в transaction flow
- Ledger imbalance (SUM DEBIT != SUM CREDIT)
- Negative balance attempt (despite CHECK constraint)
- Idempotency key collision с different payload

НЕ логировать

- Пароли, полные JWT tokens, PII (email — маскировать, логировать только userId)

10.4 Distributed Tracing

Micrometer Tracing + Jaeger. Каждый HTTP-запрос получает traceId, который пробрасывается в Kafka message headers. Позволяет проследить путь: HTTP -> Transaction -> Ledger -> Outbox -> Kafka -> Analytics consumer.

10.5 Reconciliation (сверка)

Новый процесс

Фоновый job (каждые 5 мин): сверяет accounts.balance с SUM(ledger_entries) по каждому счёту. При расхождении: ERROR лог + PagerDuty alert + блокировка счёта до ручного разбора. Это последняя линия защиты от ошибок в коде.

11. Тестирование

11.1 Типы тестов

Тип	Что тестировать	Инструмент
Unit	Бизнес-логика (TransactionService, FraudDetector, LimitChecker)	JUnit 5 + Mockito
Integration	Repository + реальная БД, REST controllers, Redis	Testcontainers (PG + Redis + Kafka)
Concurrency	Параллельные переводы, race conditions, deadlocks	ExecutorService + CountDownLatch
Contract	API контракты (request/response валидация)	Spring MockMvc
Resilience	Поведение при отказе Redis/Kafka/DB replica	Testcontainers + Toxiproxy

11.2 Критические тест-кейсы

- **Двойное списание:** отправь 2 одинаковых перевода с одним idempotency-key — должен пройти только 1
- **Race condition на балансе:** баланс 100$, запусти 10 потоков по 20$ — должно пройти ровно 5, остальные 5 — INSUFFICIENT_FUNDS
- **Deadlock:** одновременные переводы A->B и B->A — НЕ должно быть deadlock (ordered locking)
- **Ledger consistency:** сумма всех DEBIT = сумма всех CREDIT (всегда!)
- **Outbox delivery:** событие появляется в outbox ТОЛЬКО если транзакция прошла
- **Partitioning:** EXPLAIN показывает partition pruning
- **Redis fallback:** остановить Redis — система продолжает работать (деградация, не отказ)
- **Graceful shutdown:** in-flight транзакция завершается корректно при shutdown

12. Нефункциональные требования

12.1 Производительность

- **Перевод:** < 200ms (p99)
- **Список транзакций (пагинация):** < 100ms
- **Аналитика (месячный отчёт):** < 500ms
- **Бенчмарки:** JMH для критичных участков, Gatling/JMeter для нагрузочного тестирования

12.2 Безопасность

- JWT RS256 с коротким TTL (15 мин) + refresh token (30 дней) + token family rotation
- Rate limiting на Auth API + Transaction API (Bucket4j + Redis)
- Все операции с деньгами — только для аутентифицированных пользователей
- Пользователь может оперировать только своими счетами (проверка owner_id)
- Redis blacklist для revoked access tokens
- Audit log для всех значимых операций
- PII masking в логах

13. Roadmap: порядок реализации

Этап 1: Фундамент (неделя 1-2)

- Инициализация проекта: Spring Boot, Docker, Flyway, PostgreSQL, Redis
- Infrastructure layer: idempotency filter, rate limiter middleware
- Auth module: регистрация, логин, JWT RS256 + refresh + token family
- Account module: CRUD, баланс, holds
- Audit module: базовый audit log
- Базовые интеграционные тесты с Testcontainers

Этап 2: Ядро — Транзакции (неделя 3-4)

- Transaction module: переводы, ledger, идемпотентность (отдельная таблица)
- Pessimistic locking с ordered locks (предотвращение deadlock)
- Optimistic locking (для сравнения)
- Concurrency тесты (10 потоков, deadlock тест)
- EXPLAIN ANALYZE для ключевых запросов
- Reconciliation job (сверка balance vs ledger)

Этап 3: Events + Аналитика (неделя 5-6)

- Outbox pattern + Kafka (SKIP LOCKED polling)
- Analytics module: Kafka consumer, витрины, агрегации
- Partitioning: transactions, ledger_entries, audit_log
- Категоризация расходов (event consumer)
- Redis кэширование: балансы, счётчики лимитов

Этап 4: Продакшн-фичи (неделя 7-8)

- Limits module: Redis counters + SQL fallback + ночной пересчёт
- Fraud detection: правила с severity + Redis предвычисление
- Notification module: роутинг по каналам
- Circuit breaker (Resilience4j) для Redis, notification providers
- Мониторинг: Micrometer, Prometheus, Grafana дашборд, Jaeger трейсинг

Этап 5: Полировка (неделя 9-10)

- Resilience тесты (Toxiproxy: Redis down, Kafka down)
- Нагрузочное тестирование (Gatling): найти bottlenecks
- Graceful shutdown + rolling update
- GDPR: анонимизация пользователей
- Code review: убрать god-classes, проверить single responsibility
- Документация: OpenAPI, README, ADR (Architecture Decision Records)
- Профилирование JVM: async-profiler, JFR

14. Что говорить на собеседовании

14.1 Готовые формулировки

"Я реализовал систему финансовых транзакций с двойной записью. Каждая операция порождает DEBIT и CREDIT в ledger. Это позволяет в любой момент восстановить баланс из ledger и сверить с кешированным значением в accounts. У меня есть reconciliation job, который делает эту сверку каждые 5 минут."

"Для защиты от дублирования я использовал idempotency key в отдельной таблице с INSERT ON CONFLICT — это защищает от race condition, когда два retry приходят одновременно. Ключ записывается ДО начала бизнес-логики."

"Для предотвращения deadlock при встречных переводах я блокирую счета в порядке возрастания UUID. Это гарантирует единый порядок блокировки для всех транзакций."

"Система спроектирована с graceful degradation: при отказе Redis — fallback к SQL, при отказе Kafka — outbox буферизирует события. Circuit breaker через Resilience4j переключает на fallback автоматически."

14.2 Вопросы, которые зададут

Вопрос	Суть ответа
Почему не микросервисы?	Модульный монолит с чёткими границами. ACID-транзакция для переводов требует единой БД. Готов к декомпозиции: analytics и notifications — первые кандидаты.
Что будет при сбое между дебетом и кредитом?	Невозможно — оба в одной ACID-транзакции. Если COMMIT не прошёл — откат обоих.
Как масштабировать?	Read replicas для аналитики, partitioning transactions, Redis для горячих данных. Горячие аккаунты — async зачисления / sub-accounts.
Почему Kafka, а не RabbitMQ?	Kafka: ordering по partition key, replay, log retention. RabbitMQ: проще, push-модель. Для финтеха ordering критичен.
Как обеспечить exactly-once?	Outbox + идемпотентные consumers. At-least-once + idempotency = effectively exactly-once.
Что будет если Redis упадёт?	Graceful degradation: лимиты через SQL, баланс из DB, rate limiting — skip. Circuit breaker автоматически переключает.
Как защищаетесь от deadlocks?	Ordered locking: всегда блокируем счета в порядке возрастания UUID. Deadlock невозможен по определению.
Как обеспечить целостность данных?	4 уровня: 1) DB CHECK constraints, 2) optimistic/pessimistic locking, 3) idempotency keys, 4) reconciliation job.

15. Риски системы и критические места

15.1 Таблица рисков

#	Риск	Severity	Mitigation
1	Hot account contention (мерчант, тысячи TPS)	CRITICAL	Pessimistic + ordered locks. При экстремальной нагрузке: async зачисления / sub-accounts.
2	Ledger/balance drift (расхождение данных)	CRITICAL	Reconciliation job каждые 5 мин. Alert + блокировка при расхождении.
3	Outbox lag при сбое Kafka	HIGH	Мониторинг, alert при > 1000 pending. Bounded outbox (max 10K events).
4	Redis failure — каскадная деградация	HIGH	Circuit breaker, DB fallback для всех Redis-зависимых операций.
5	Partition not created — INSERT fails	HIGH	pg_partman или cron для автосоздания. Alert при отсутствии партиции.
6	Fraud false positive — блокировка невиновного	HIGH	Severity levels. Автоблокировка только при HIGH + confirmation.
7	UUID v4 index bloat	MEDIUM	UUID v7 для всех PK (timestamp-sortable, оптимально для B-tree).
8	Idempotency race condition	MEDIUM	Отдельная таблица + INSERT ON CONFLICT DO NOTHING.
9	GDPR compliance — нет удаления	MEDIUM	Анонимизация PII вместо удаления. Status = ANONYMIZED.
10	In-flight транзакции при деплое	HIGH	Graceful shutdown: drain connections, wait for outbox flush, preStop hook.

15.2 Что продумать перед кодом

4. Определить SLA: p99 latency перевода, допустимый downtime, RPO/RTO
20. Определить capacity: сколько TPS на один account (определяет стратегию блокировок)
21. UUID v7 vs v4 — влияет на ВСЕ таблицы, решить до первой миграции
22. Error taxonomy — единый каталог кодов ошибок ДО написания контроллеров
23. CI с Testcontainers — concurrency тесты зелёные с первого дня
24. Скрипт автосоздания партиций — нельзя забыть
25. Стратегия деплоя — rolling update с drain, graceful shutdown
26. Retention policy — сколько хранить online, когда архивировать
27. Observability stack до бизнес-логики — structured logging, tracing, metrics
28. Определить контракт audit log — какие действия логировать, формат

Удачи. Этот проект — твой билет в бигтех. Делай, ломай, чини, объясняй.